DATABASE TECHNOLOGIES LAB SHEET – 6

Course Code: MCA105P

SECTION 1: DATABASE SETUP

Step 1: Create Database

```
CREATE DATABASE CollegeDB;
USE CollegeDB;
```

Step 2: Create Department Table

```
CREATE TABLE Department (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL,
   Location VARCHAR(50)
);
```

Step 3: Create Student Table

```
CREATE TABLE Student (
   StudentID INT PRIMARY KEY,
   StudentName VARCHAR(50) NOT NULL,
   Age INT,
   Gender VARCHAR(10),
   Marks DECIMAL(5,2),
   DeptID INT,
   Email VARCHAR(100),
   FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
);
```

Step 4: Create Course Table

```
CREATE TABLE Course (
   CourseID INT PRIMARY KEY,
   CourseName VARCHAR(50) NOT NULL,
   Credits INT
);
```

Step 5: Insert Department Data

```
INSERT INTO Department
VALUES
(101, 'Computer Science', 'Block A'),
(102, 'Pharmacy', 'Block B'),
(103, 'Commerce', 'Block C'),
(104, 'Mathematics', 'Block D');
```

Step 6: Insert Student Data

```
INSERT INTO Student
VALUES
(1, 'Amit', 20, 'Male', 85.50, 101, 'amit@gmail.com'),
(2, 'Priya', 21, 'Female', 92.00, 102, 'priya@gmail.com'),
(3, 'Rahul', 19, 'Male', 76.50, 101, 'rahul@gmail.com'),
(4, 'Neha', 22, 'Female', 88.00, 103, 'neha@gmail.com'),
(5, 'Arjun', 20, 'Male', 69.50, 104, 'arjun@gmail.com'),
(6, 'Sneha', 21, 'Female', 95.00, 102, 'sneha@gmail.com'),
(7, 'Rohit', 23, 'Male', 72.00, 103, 'rohit@gmail.com'),
(8, 'Pooja', 20, 'Female', 81.00, 104, 'pooja@gmail.com');
```

Step 7: Insert Course Data

```
INSERT INTO Course
VALUES
(201, 'Database Management System', 4),
(202, 'Python Programming', 4),
(203, 'Mathematics', 3),
(204, 'Computer Networks', 3),
(205, 'Data Structures', 4);
```

Step 8: Display All Tables

```
SELECT * FROM Department;
SELECT * FROM Student;
SELECT * FROM Course;
```

SECTION 2: STORED PROCEDURES

Q1. Display All Students

Question: Create a stored procedure named ShowStudents to display all records from the Student table.

```
DELIMITER $$

CREATE PROCEDURE ShowStudents()
BEGIN
   SELECT * FROM Student;
END $$

DELIMITER ;

CALL ShowStudents();
```

Q2. Display All Departments

Question: Create a stored procedure named ShowDepartments to display all records from the Department table.

```
DELIMITER $$

CREATE PROCEDURE ShowDepartments()
```

```
BEGIN
   SELECT * FROM Department;
END $$

DELIMITER ;

CALL ShowDepartments();
```

Q3. Display All Courses

Question: Create a stored procedure named ShowCourses to display all records from the Course table.

```
DELIMITER $$

CREATE PROCEDURE ShowCourses()
BEGIN
   SELECT * FROM Course;
END $$

DELIMITER ;

CALL ShowCourses();
```

Q4. Display Student by ID

Question: Create a stored procedure named GetStudent that accepts StudentID as an IN parameter and displays that student's details. Test it using StudentID 3.

```
DELIMITER $$

CREATE PROCEDURE GetStudent(IN p_StudentID INT)
BEGIN
   SELECT *
   FROM Student
   WHERE StudentID = p_StudentID;
END $$

DELIMITER ;

CALL GetStudent(3);
```

Q5. Display Students from a Department

Question: Create a stored procedure named GetStudentsByDept that accepts DeptID as an IN parameter and displays students belonging to that department. Test it using DeptID 102.

```
DELIMITER $$

CREATE PROCEDURE GetStudentsByDept(IN p_DeptID INT)
BEGIN
   SELECT *
   FROM Student
   WHERE DeptID = p_DeptID;
END $$
```

```
DELIMITER ;

CALL GetStudentsByDept(102);
```

Q6. Display Students Above Given Marks

Question: Create a stored procedure named StudentsAboveMarks that accepts marks as an IN parameter and displays students whose marks are greater than the given value. Test it using 80.

```
DELIMITER $$

CREATE PROCEDURE StudentsAboveMarks(IN p_Marks DECIMAL(5,2))
BEGIN
   SELECT *
   FROM Student
   WHERE Marks > p_Marks;
END $$

DELIMITER ;

CALL StudentsAboveMarks(80);
```

Q7. Display Total Number of Students

Question: Create a stored procedure named TotalStudents that calculates and displays the total number of students in the Student table.

```
DELIMITER $$

CREATE PROCEDURE TotalStudents()
BEGIN
   SELECT COUNT(*) AS TotalStudents
   FROM Student;
END $$

DELIMITER ;

CALL TotalStudents();
```

Q8. Check Student Result

Question: Create a stored procedure named CheckResult that accepts StudentID as an IN parameter. If marks are 50 or above, display Pass; otherwise, display Fail.

```
DELIMITER $$

CREATE PROCEDURE CheckResult(IN p_StudentID INT)
BEGIN
   DECLARE student_marks DECIMAL(5,2);

   SELECT Marks INTO student_marks
   FROM Student
```

```
    WHERE StudentID = p_StudentID;

    IF student_marks IS NULL THEN
        SELECT 'Student not found or marks are NULL' AS Result;
    ELSEIF student_marks >= 50 THEN
        SELECT 'Pass' AS Result;
    ELSE
        SELECT 'Fail' AS Result;
    END IF;
END $$

DELIMITER ;

CALL CheckResult(3);
```

Q9. Display Students of a Particular Gender

Question: Create a stored procedure named GetStudentsByGender that accepts Gender as an IN parameter and displays students of that gender. Test it using Female.

```
DELIMITER $$

CREATE PROCEDURE GetStudentsByGender(IN p_Gender VARCHAR(10))
BEGIN
    SELECT *
    FROM Student
    WHERE Gender = p_Gender;
END $$

DELIMITER ;

CALL GetStudentsByGender('Female');
```

Q10. Display Students Within a Marks Range

Question: Create a stored procedure named StudentsBetweenMarks that accepts two marks values and displays students whose marks are between those values. Test it using 70 and 90.

```
DELIMITER $$

CREATE PROCEDURE StudentsBetweenMarks(
    IN p_MinMarks DECIMAL(5,2),
    IN p_MaxMarks DECIMAL(5,2)
)
BEGIN
    SELECT *
    FROM Student
    WHERE Marks BETWEEN p_MinMarks AND p_MaxMarks;
END $$

DELIMITER ;

CALL StudentsBetweenMarks(70, 90);
```

SECTION 3: FUNCTIONS

Q11. Add Two Numbers

Question: Create a function named AddTwoNumbers that accepts two integers and returns their sum. Test it with 10 and 20.

```
DELIMITER $$

CREATE FUNCTION AddTwoNumbers(
   a INT,
   b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
   RETURN a + b;
END $$

DELIMITER ;

SELECT AddTwoNumbers(10, 20) AS Sum;
```

Q12. Find Square

Question: Create a function named FindSquare that accepts an integer and returns its square. Test it with 5.

```
DELIMITER $$

CREATE FUNCTION FindSquare(n INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
   RETURN n * n;
END $$

DELIMITER ;

SELECT FindSquare(5) AS Square;
```

Q13. Check Pass or Fail

Question: Create a function named ResultStatus that accepts marks and returns Pass if marks are 50 or above; otherwise, it returns Fail.

```
DELIMITER $$

CREATE FUNCTION ResultStatus(p_Marks DECIMAL(5,2))
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
   IF p_Marks >= 50 THEN
      RETURN 'Pass';
```

```
   ELSE
      RETURN 'Fail';
   END IF;
END $$

DELIMITER ;

SELECT ResultStatus(75) AS Result;
SELECT ResultStatus(35) AS Result;
```

Q14. Find Student Grade

Question: Create a function named StudentGrade that accepts marks and returns:

- A for 90 or above
- B for 75–89
- C for 60–74
- D for below 60

```
DELIMITER $$

CREATE FUNCTION StudentGrade(p_Marks DECIMAL(5,2))
RETURNS CHAR(1)
DETERMINISTIC
NO SQL
BEGIN
   IF p_Marks >= 90 THEN
      RETURN 'A';
   ELSEIF p_Marks >= 75 THEN
      RETURN 'B';
   ELSEIF p_Marks >= 60 THEN
      RETURN 'C';
   ELSE
      RETURN 'D';
   END IF;
END $$

DELIMITER ;

SELECT StudentGrade(95) AS Grade;
SELECT StudentGrade(80) AS Grade;
SELECT StudentGrade(65) AS Grade;
SELECT StudentGrade(40) AS Grade;
```

Q15. Find Age Category

Question: Create a function named AgeCategory that accepts age and returns Adult if age is 18 or above; otherwise, it returns Minor.

```
DELIMITER $$

CREATE FUNCTION AgeCategory(p_Age INT)
RETURNS VARCHAR(10)
DETERMINISTIC
NO SQL
BEGIN
```

```
   IF p_Age >= 18 THEN
      RETURN 'Adult';
   ELSE
      RETURN 'Minor';
   END IF;
END $$

DELIMITER ;

SELECT AgeCategory(20) AS Category;
SELECT AgeCategory(15) AS Category;
```

Q16. Calculate Bonus

Question: Create a function named CalculateBonus that accepts marks and returns:

- 1000 for 90 or above
- 500 for 75–89
- 200 for below 75

```
DELIMITER $$

CREATE FUNCTION CalculateBonus(p_Marks DECIMAL(5,2))
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
   IF p_Marks >= 90 THEN
      RETURN 1000;
   ELSEIF p_Marks >= 75 THEN
      RETURN 500;
   ELSE
      RETURN 200;
   END IF;
END $$

DELIMITER ;

SELECT CalculateBonus(95) AS Bonus;
SELECT CalculateBonus(80) AS Bonus;
SELECT CalculateBonus(60) AS Bonus;
```

Q17. Find Higher Number

Question: Create a function named HigherNumber that accepts two integers and returns the greater number.

```
DELIMITER $$

CREATE FUNCTION HigherNumber(
   a INT,
   b INT
)
RETURNS INT
DETERMINISTIC
NO SQL
```

```
BEGIN
   IF a > b THEN
      RETURN a;
   ELSE
      RETURN b;
   END IF;
END $$

DELIMITER ;

SELECT HigherNumber(25, 40) AS HigherNumber;
```

Q18. Calculate Double Marks

Question: Create a function named DoubleMarks that accepts marks and returns twice the marks.

```
DELIMITER $$

CREATE FUNCTION DoubleMarks(p_Marks DECIMAL(5,2))
RETURNS DECIMAL(6,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_Marks * 2;
END $$

DELIMITER ;

SELECT DoubleMarks(45) AS DoubleMarks;
```

Q19. Calculate Age After Five Years

Question: Create a function named AgeAfterFiveYears that accepts current age and returns the age after five years.

```
DELIMITER $$

CREATE FUNCTION AgeAfterFiveYears(p_Age INT)
RETURNS INT
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_Age + 5;
END $$

DELIMITER ;

SELECT AgeAfterFiveYears(20) AS FutureAge;
```

Q20. Display Student Name, Marks and Grade

Question: Using the StudentGrade() function, write a SELECT query to display StudentName, Marks and Grade for every student in the Student table.

```
SELECT
   StudentName,
   Marks,
   StudentGrade(Marks) AS Grade
FROM Student;
```